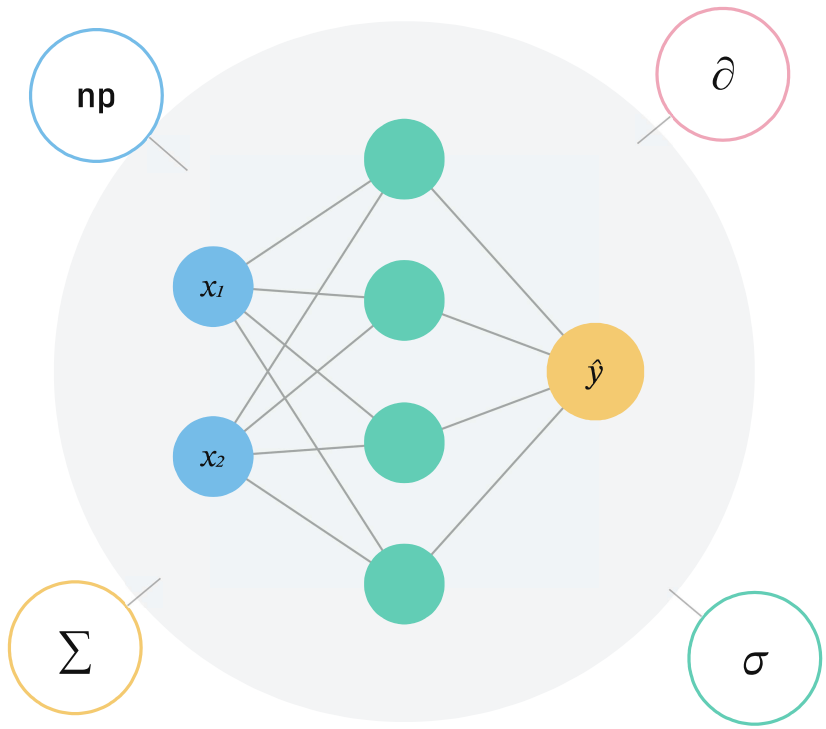
MÓDULO 6 · MACHINE LEARNING NA MÃO, COM NUMPY

Aula 6.12

Perceptron e MLP com Backpropagation

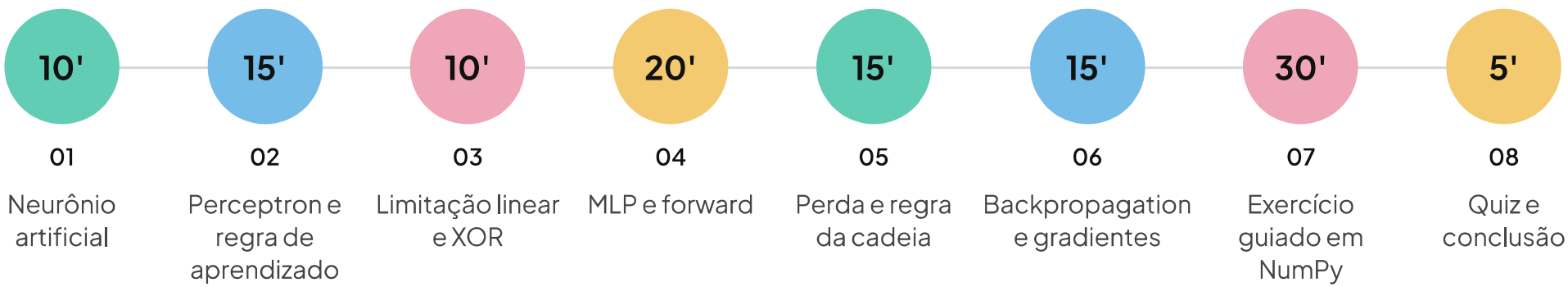
Do neurônio único à rede neural que aprende o XOR.

NumPy · Matplotlib · 2 horas



AULA 6.12 · 120 MINUTOS

Roteiro da aula



Neurônio → Perceptron → Limitação linear → XOR → MLP → Forward → Perda → Regra da cadeia → Backpropagation → Gradiente → Atualização → Aprendizado

01

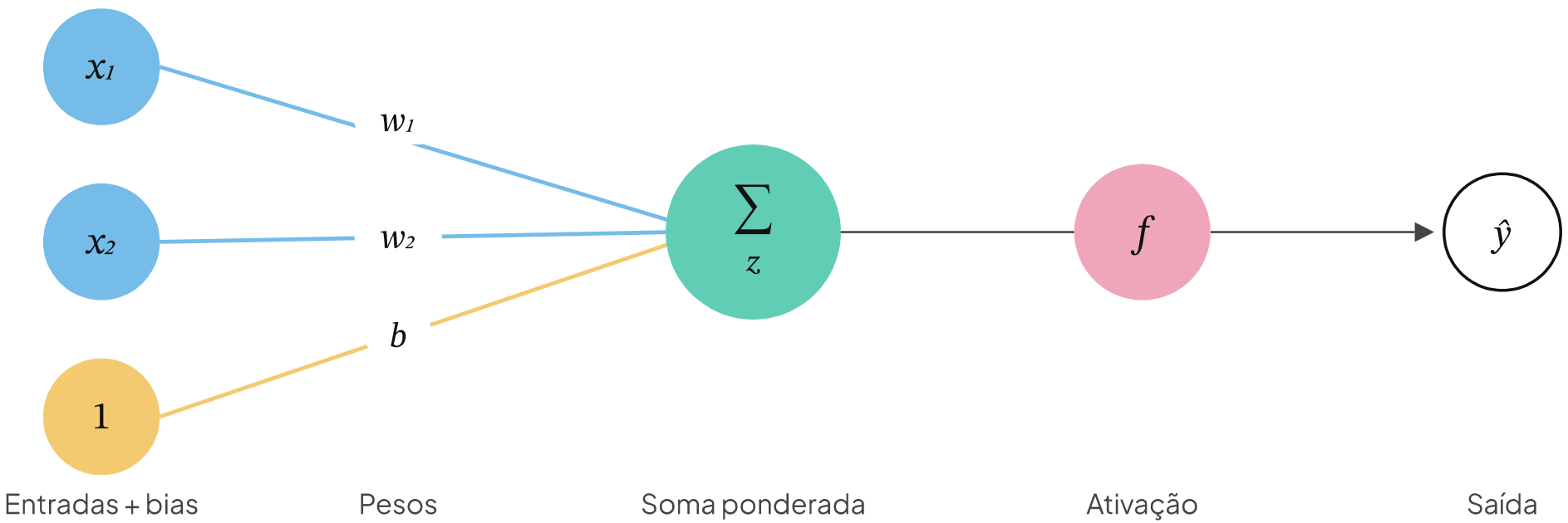
SEÇÃO 1 · 10 MINUTOS

O neurônio artificial

Como uma máquina consegue aprender relações entre entradas e saídas?

BLOCO 1 · INTRODUÇÃO

Anatomia de um neurônio artificial



Inspiração biológica: o neurônio integra sinais e decide se responde. O modelo matemático guarda só essa ideia: combinar entradas com pesos, somar um bias e aplicar uma função.

BLOCO 2 · O NEURÔNIO ARTIFICIAL

Soma ponderada

$$z = w_1 x_1 + w_2 x_2 + b$$

x **x_1, x_2**
entradas da amostra

w **w_1, w_2**
pesos: importância e sentido de cada entrada

b **b**
bias: deslocamento que não depende das entradas

z **z**
soma ponderada: um único número

$$z = \mathbf{w}^{\mathrm{T}}\mathbf{x} + b = \begin{pmatrix} w_1 & w_2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + b$$

Produto escalar: multiplica componente a componente e soma. A forma vetorial vale para qualquer número de entradas.

BLOCO 2 · EXEMPLO NUMÉRICO

Calculando uma soma ponderada

$$\mathbf{x} = (1; 0)$$

$$\mathbf{w} = (0{,}8; -0{,}4)$$

$$b = -0{,}2$$

O ponto e vírgula separa componentes; a vírgula é a casa decimal.

$w_1 x_1$	$0{,}8 \cdot 1$	$0{,}8$
$w_2 x_2$	$-0{,}4 \cdot 0$	0
b		$-0{,}2$
z	positivo: as entradas superam o bias	$0{,}6$

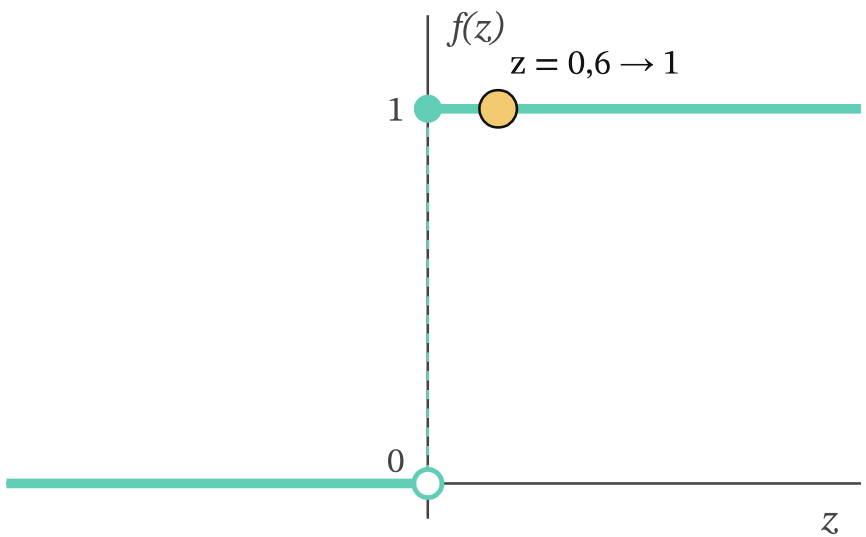
```
x = np.array([1, 0])
w = np.array([0.8, -0.4])
b = -0.2
z = np.dot(w, x) + b   # 0.6000000000000001 (ponto flutuante)
```

BLOCO 3 · ATIVAÇÃO

Função de ativação degrau

$$f(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}$$

- **Valor interno** $z = 0{,}6$ é contínuo.
- **Saída** $f(0{,}6) = 1$ é uma classe.
- Versão binária didática da ativação do Perceptron.



02

SEÇÃO 2 · 15 MINUTOS

O Perceptron

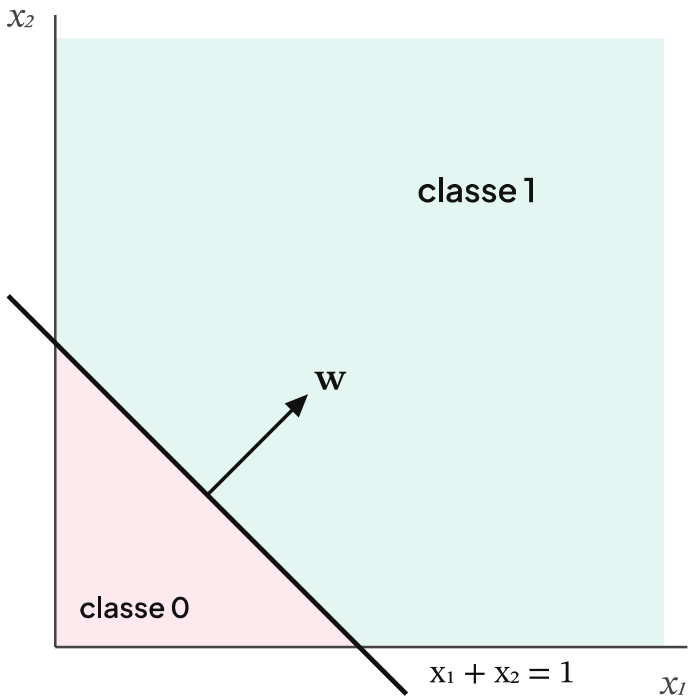
Um único neurônio, uma fronteira linear e uma regra de aprendizado.

Perceptron e fronteira de decisão

$$w_1 x_1 + w_2 x_2 + b = 0$$

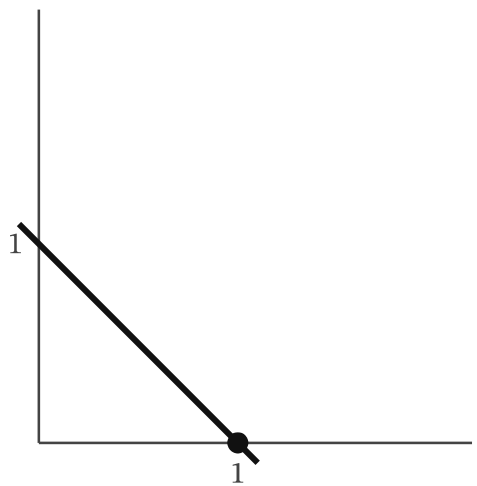
$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2} \quad (w_2 \neq 0)$$

- $z \geq 0$: previsão classe 1
- $z < 0$: previsão classe 0
- Em 2D a fronteira é uma reta; em mais dimensões, um hiperplano.



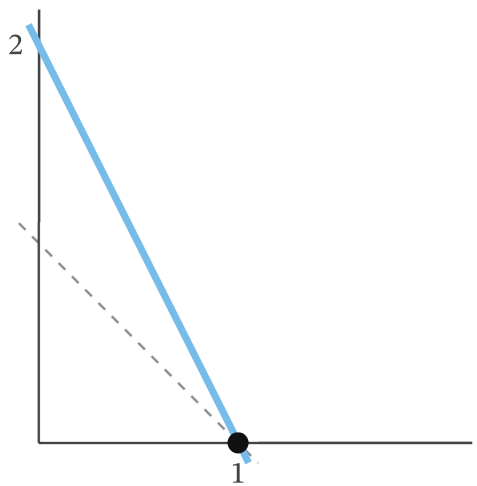
BLOCO 4 · GEOMETRIA

Efeito dos pesos e do bias na reta



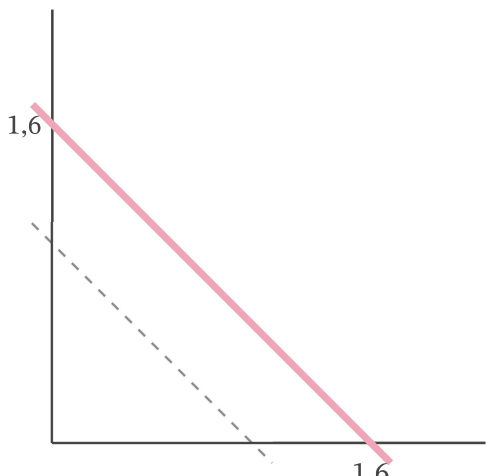
$\mathbf{w} = (1; 1), b = -1$

Reta de referência



$\mathbf{w} = (1; 0{,}5), b = -1$

Pesos mudam a inclinação



$\mathbf{w} = (1; 1), b = -1{,}6$

Bias desloca a reta

BLOCO 5 · APRENDIZADO

Ciclo de aprendizado do Perceptron



BLOCO 5 · REGRA CLÁSSICA, RÓTULOS −1 E +1

Regra de atualização do Perceptron

$$\hat{y}_i = \text{sign}(\mathbf{w}^{\mathrm{T}}\mathbf{x}_i + b)$$

Se $\hat{y}_i \neq y_i$ (amostra errada):

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \mathbf{x}_i$$

$$b \leftarrow b + \eta\, y_i$$

η taxa de aprendizado (> 0)

y_i rótulo verdadeiro, −1 ou +1

$\mathbf{x}_i$ vetor de entradas da amostra

$\mathbf{w}$ vetor de pesos

b bias

sign +1 se z ≥ 0, senão −1

Relação com o degrau 0/1 do slide 7: sign(z) = 2·f(z) − 1. Mesma decisão; só muda a codificação dos rótulos.

Por que a correção ajuda

Acerto quando $y_i z_i > 0$. Após a atualização:

$$y_i z'_i = y_i z_i + \eta\, y_i^2 (\|\mathbf{x}_i\|^2 + 1)$$

Como $y_i^2 = 1$, o termo somado é sempre positivo: a amostra é empurrada para o lado correto da reta.

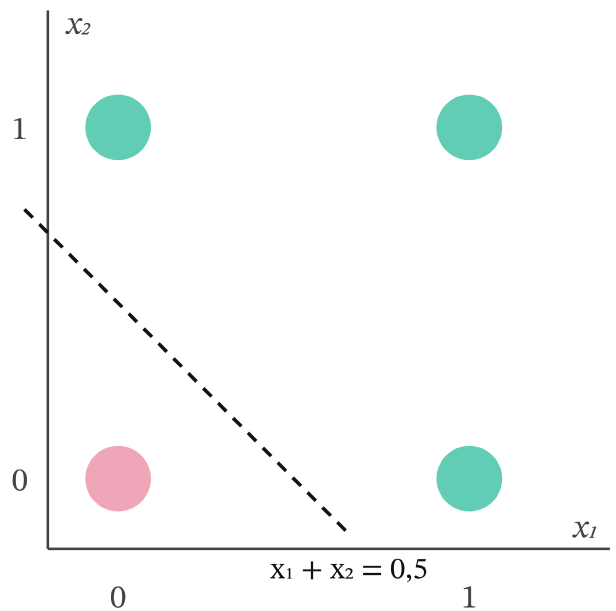
Convergência: garantida em número finito de atualizações apenas se os dados forem linearmente separáveis.

BLOCO 6 · EXEMPLO

Função OR com rótulos −1 e +1

x_1	x_2	OR	y (Perceptron)
0	0	0	−1
0	1	1	+1
1	0	1	+1
1	1	1	+1

$0 \rightarrow -1 \qquad 1 \rightarrow +1$



BLOCO 6 · RASTREAMENTO MANUAL

Treinamento do OR passo a passo

Época	x_i	y_i	z	$\hat{y}_i$	Erro?	w depois	b depois
1	(0; 0)	−1	0	+1	sim	(0; 0)	−0,1
	(0; 1)	+1	−0,1	−1	sim	(0; 0,1)	0
	(1; 0)	+1	0	+1	não	(0; 0,1)	0
	(1; 1)	+1	0,1	+1	não	(0; 0,1)	0
2	(0; 0)	−1	0	+1	sim	(0; 0,1)	−0,1
	(0; 1)	+1	0	+1	não	(0; 0,1)	−0,1
	(1; 0)	+1	−0,1	−1	sim	(0,1; 0,1)	0
	(1; 1)	+1	0,2	+1	não	(0,1; 0,1)	0
3	(0; 0)	−1	0	+1	sim	(0,1; 0,1)	−0,1
	(0; 1), (1; 0), (1; 1): z = 0; 0; 0,1 → todas corretas						
4	(0; 0): z = −0,1 → −1 ✓ · demais corretas · 0 erros → fim						

$$\mathbf{w} = (0{,}1;\ 0{,}1)$$

$$b = -0{,}1$$

4 épocas

Fronteira: $x_1 + x_2 = 1$

(0; 1) e (1; 0) ficam sobre a reta (z = 0) e recebem +1 pela convenção z ≥ 0.

BLOCO 6 · IMPLEMENTAÇÃO

Perceptron em NumPy

```python
def treinar_perceptron(X, y, taxa_aprendizado=0.1, max_epocas=100):
    pesos = np.zeros(X.shape[1])
    bias = 0.0

    for epoca in range(max_epocas):
        erros = 0

        for xi, yi in zip(X, y):
            z = np.dot(xi, pesos) + bias

            previsao = 1 if z >= 0 else -1

            if previsao != yi:
                pesos += taxa_aprendizado * yi * xi
                bias += taxa_aprendizado * yi
                erros += 1

        if erros == 0:
            break

    return pesos, bias, epoca + 1
```

np.zeros · w e b começam em zero

np.dot(xi, pesos) + bias · $z = w^T x_i + b$

1 if z >= 0 else -1 · função sinal

pesos += … · $w \leftarrow w + \eta\, y_i\, x_i$, só quando erra

erros == 0 · época sem erros encerra o laço

Rastreamento manual = execução de referência

```
pesos = [0.1 0.1]
bias = -0.1
épocas = 4
```

03

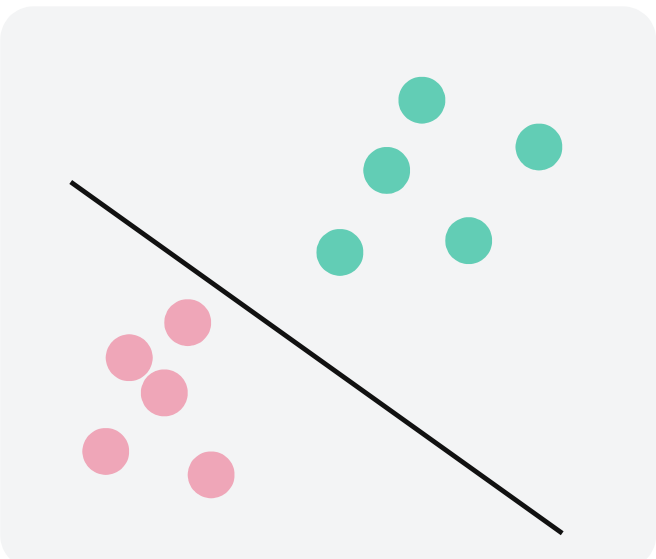
SEÇÃO 3 · 10 MINUTOS

Limitação linear e XOR

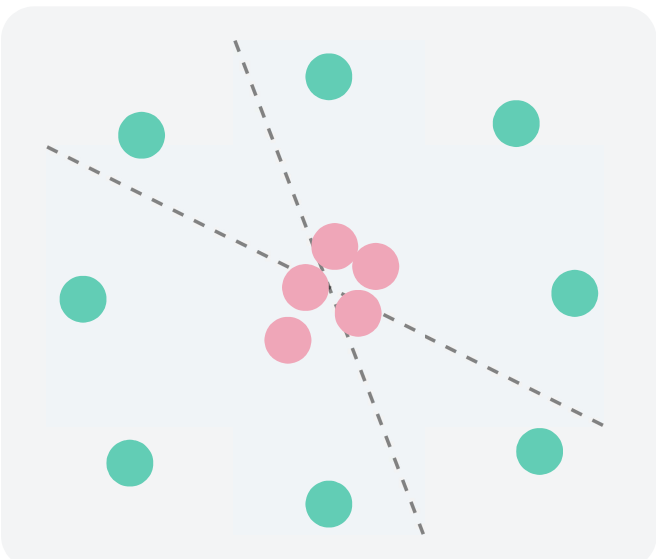
O que uma única reta não consegue separar.

BLOCO 7 · LIMITAÇÕES

Separabilidade linear



Separável por uma reta



Nenhuma reta separa

O Perceptron só representa fronteiras lineares: ele classifica perfeitamente apenas dados linearmente separáveis.

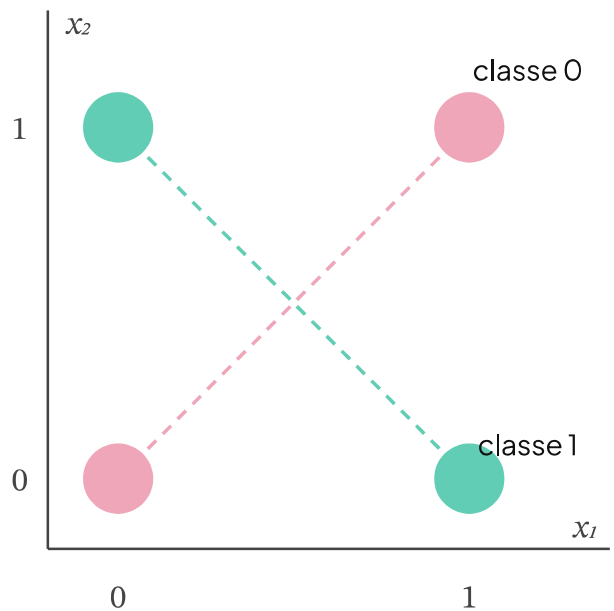
BLOCO 8 · O PROBLEMA XOR

Função XOR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

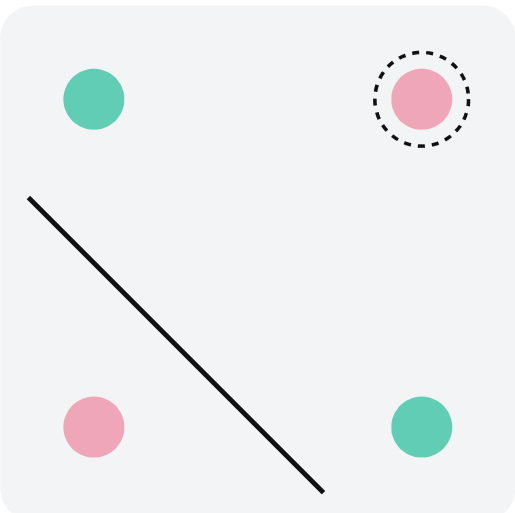
Entradas diferentes → 1

Entradas iguais → 0



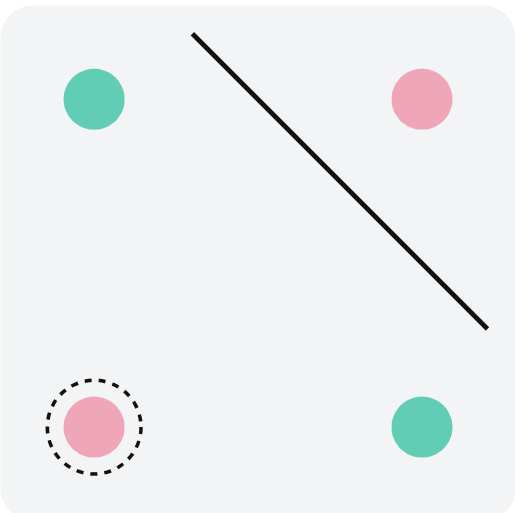
BLOCO 9 · DEMONSTRAÇÃO VISUAL

Tentativas de separar o XOR com uma reta



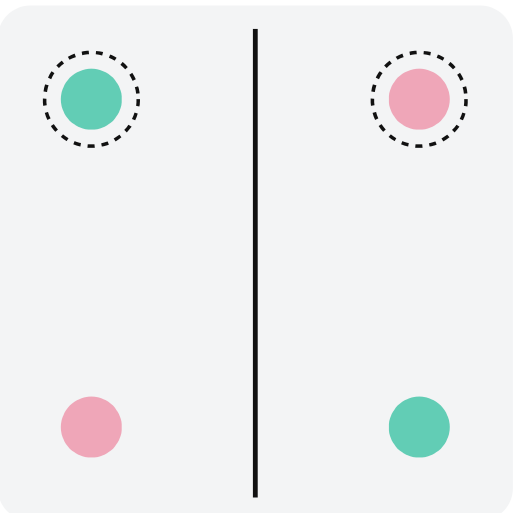
$x_1 + x_2 = 0{,}5$

1 erro: (1; 1)



$x_1 + x_2 = 1{,}5$

1 erro: (0; 0)



$x_1 = 0{,}5$

2 erros

O Perceptron de uma única camada não consegue aprender a função XOR.

BLOCO 10 · PROVA POR CONTRADIÇÃO

Demonstração algébrica da impossibilidade

Condições necessárias

(1)	$(0;0)$	$b < 0$
(2)	$(1;0)$	$w_1 + b \geq 0$
(3)	$(0;1)$	$w_2 + b \geq 0$
(4)	$(1;1)$	$w_1 + w_2 + b < 0$

Dedução

$w_1 \geq -b$	de (2)
$w_2 \geq -b$	de (3)
$w_1 + w_2 + b \geq -b$	somando e adicionando b
$-b > 0 \Rightarrow w_1 + w_2 + b > 0$	por (1)

Contradição com (4). Nenhuma escolha de w_1, w_2 e b satisfaz as quatro condições.

04

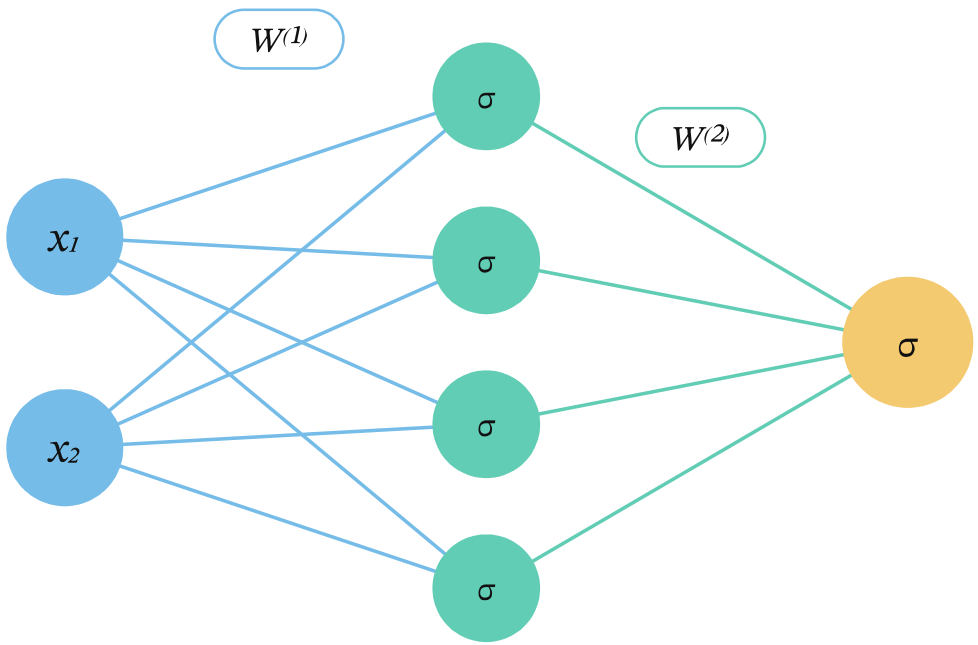
SEÇÃO 4 · 20 MINUTOS

MLP e forward propagation

Mais de um neurônio, organizados em camadas.

BLOCO 11 · MULTILAYER PERCEPTRON

Arquitetura 2 → 4 → 1



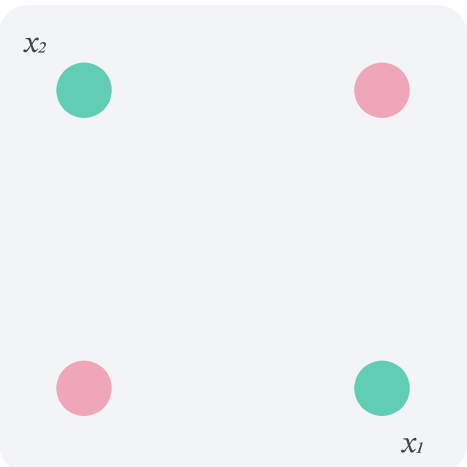
Entrada
sem parâmetros

Oculta
camada treinável 1

Saída
camada treinável 2

$W^{(1)}$ 2×4	8 pesos
$b^{(1)}$ 1×4	4 biases
$W^{(2)}$ 4×1	4 pesos
$b^{(2)}$ 1×1	1 bias
Total	**17 parâmetros**

Camada oculta como nova representação



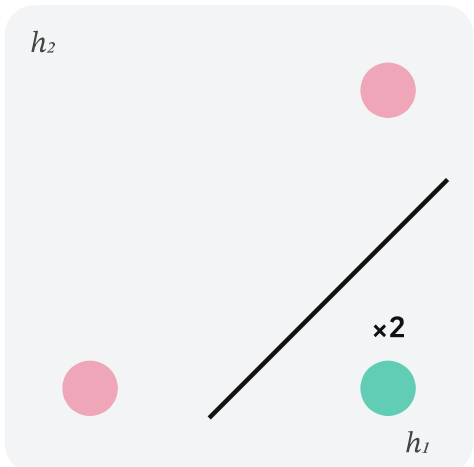
Espaço original

$h_1 = \sigma(20x_1 + 20x_2 - 10) \approx \text{OR}$

$h_2 = \sigma(20x_1 + 20x_2 - 30) \approx \text{AND}$

$\hat{y} = \sigma(20h_1 - 20h_2 - 10)$

Pesos escolhidos à mão para ilustrar, com 2 neurônios. Na rede treinada, os 4 neurônios ocultos aprendem as próprias combinações a partir de pesos aleatórios.



Espaço oculto: separável

BLOCO 13 · ATIVAÇÃO DIFERENCIÁVEL

Função sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

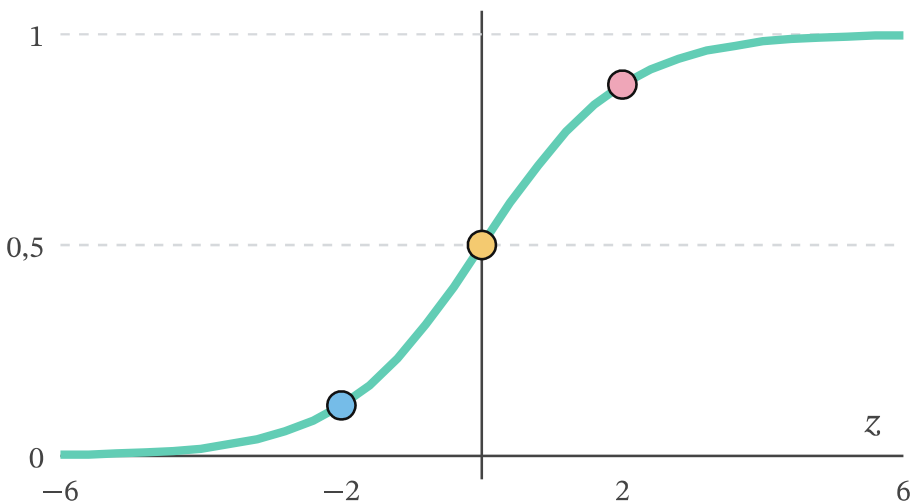
$e \approx 2{,}718$, base do logaritmo natural

z soma ponderada do neurônio

$\sigma(z)$ ativação, sempre entre 0 e 1

$\sigma(-2)$	$\sigma(0)$	$\sigma(2)$
0,119	**0,5**	**0,881**

Ativação entre 0 e 1 não é automaticamente uma probabilidade calibrada.



BLOCO 14 · DERIVADA

Derivada da sigmoid

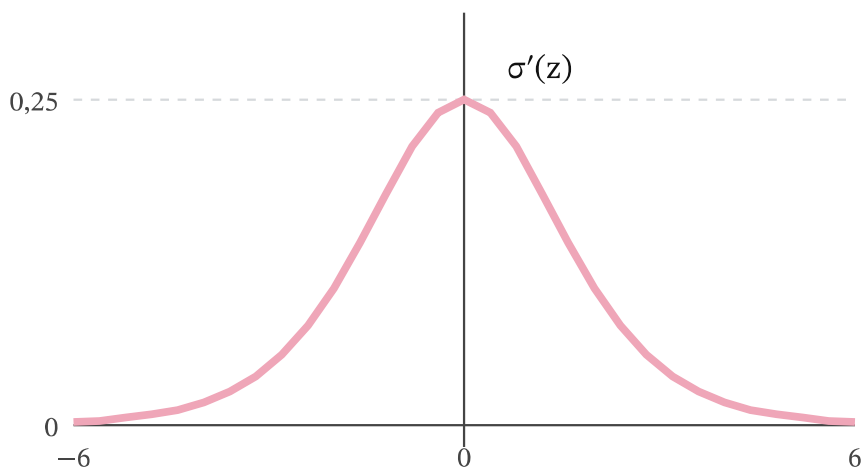
$$\sigma(z) = (1 + e^{-z})^{-1}$$

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

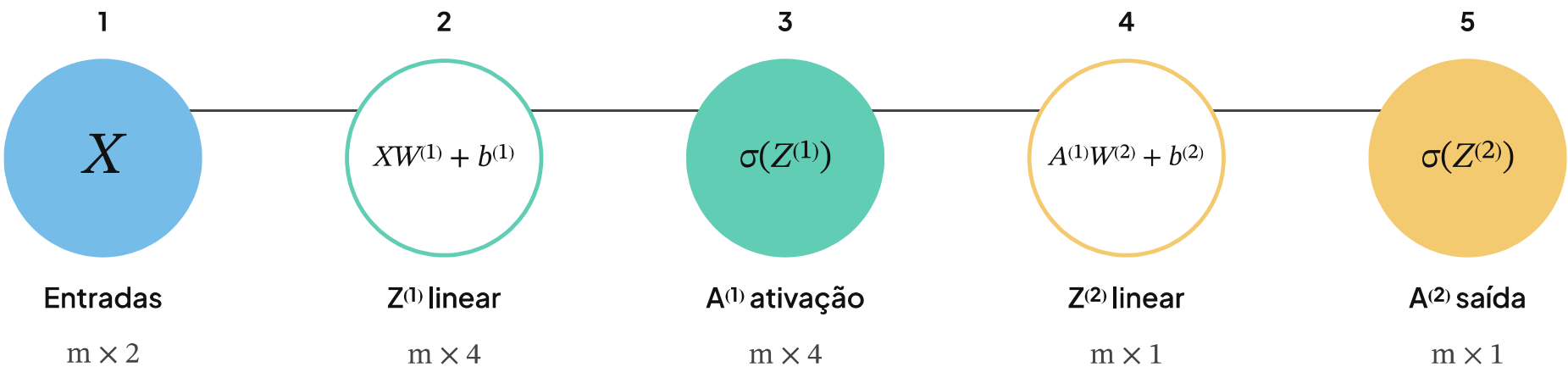
$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Com $a = \sigma(z)$ já calculado no forward: $\sigma' = a(1 - a)$. Exemplos: $\sigma'(0) = 0{,}25 \cdot \sigma'(2) \approx 0{,}105$.

Degrau: derivada zero em quase todo ponto, sem informação para ajustar pesos.



Forward propagation em cinco passos



BLOCO 16 · CAMADA OCULTA

Primeira camada: $Z^{(1)}$ e $A^{(1)}$

$$Z^{[1]} = XW^{[1]} + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

X entradas: linha = amostra, coluna = variável

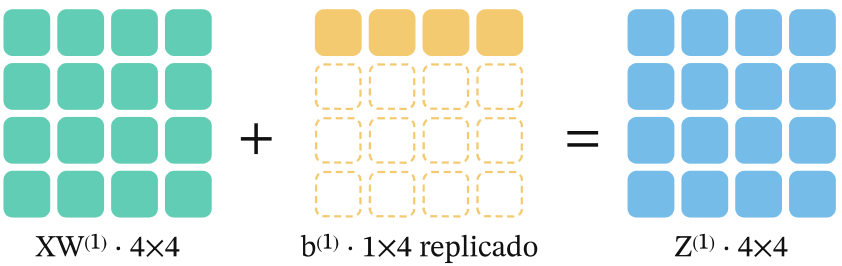
$W^{(1)}$ linha k = entrada k, coluna j = neurônio j

$b^{(1)}$ um bias por neurônio oculto

$Z^{(1)}$ somas ponderadas · $A^{(1)}$ ativações

$$z_{ij} = x_{i1}w_{1j} + x_{i2}w_{2j} + b_j$$

Broadcasting do bias



A linha de biases é repetida para cada amostra: cada neurônio soma o mesmo bias em todas as linhas.

BLOCO 17 · CAMADA DE SAÍDA

Camada de saída: $Z^{(2)}$ e $A^{(2)}$

$$Z^{[2]} = A^{[1]} W^{[2]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

$$z^{[2]}_i = \sum_{j=1..4} a^{[1]}_{ij} \, w^{[2]}_j + b^{[2]}$$

- $\mathbf{A^{(1)}}$ ativações ocultas: entrada desta camada
- $\mathbf{W^{(2)}}$ um peso por neurônio oculto
- $\mathbf{b^{(2)}}$ bias da saída
- $\mathbf{A^{(2)}}$ saída final: valor estimado para a classe 1 em cada linha do XOR
- Classe prevista: 1 se $A^{(2)} \geq 0{,}5$, senão 0

BLOCO 18 · FORMATOS DE TODOS OS ARRAYS

Dimensões das matrizes

Elemento	Dimensão	XOR
X	$\mathrm{m} \times 2$	4×2
$W1$	2×4	2×4
$b1$	1×4	1×4
$Z1$	$\mathrm{m} \times 4$	4×4
$A1$	$\mathrm{m} \times 4$	4×4
$W2$	4×1	4×1
$b2$	1×1	1×1
$Z2$	$\mathrm{m} \times 1$	4×1
$A2$	$\mathrm{m} \times 1$	4×1

$$(a \times \mathbf{k})(\mathbf{k} \times c) = (a \times c)$$

$X\,(\mathrm{m}\times 2) \cdot W1\,(2\times 4) \rightarrow \mathrm{m}\times 4$

$+\, b1\,(1\times 4)$ por broadcasting $\rightarrow Z1\ \mathrm{m}\times 4$

$A1\,(\mathrm{m}\times 4) \cdot W2\,(4\times 1) \rightarrow \mathrm{m}\times 1$

$+\, b2\,(1\times 1)$ por broadcasting $\rightarrow Z2\ \mathrm{m}\times 1$

Dados: linha = amostra. Pesos: linha = o que entra, coluna = o que sai. A sigmoid preserva o formato.

BLOCO 19 · IMPLEMENTAÇÃO

Forward em NumPy

```python
def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))


def forward(X, params):
    Z1 = X @ params["W1"] + params["b1"]
    A1 = sigmoid(Z1)

    Z2 = A1 @ params["W2"] + params["b2"]
    A2 = sigmoid(Z2)

    cache = {
        "X": X,
        "Z1": Z1,
        "A1": A1,
        "Z2": Z2,
        "A2": A2
    }

    return A2, cache
```

np.clip · evita overflow em np.exp

@ · multiplicação matricial

+ params["b1"] · broadcasting do bias

sigmoid(Z1) · elemento a elemento

cache · valores intermediários que o backward vai reutilizar

Formatos com X do XOR

```
Z1 (4, 4) A1 (4, 4)
Z2 (4, 1) A2 (4, 1)
```

05

SEÇÃO 5 · 15 MINUTOS

Perda e regra da cadeia

Medir o erro e descobrir de onde ele vem.

BLOCO 20 · FUNÇÃO DE PERDA

Binary cross-entropy

$$L = -\frac{1}{m}\sum_{i=1}^{m}\left[\, y_i \log(a_i) + (1 - y_i)\log(1 - a_i)\,\right]$$

m número de amostras; 1/m tira a média

y_i rótulo verdadeiro, 0 ou 1

a_i previsão da rede, entre 0 e 1

log logaritmo natural (base e)

y = 1 · sobra $-\log(a_i)$

y = 0 · sobra $-\log(1 - a_i)$

BLOCO 21 · EXEMPLO NUMÉRICO

Exemplo numérico da BCE

y = 1 · a = 0,8

$$L = -\log(0{,}8) \approx 0{,}223$$

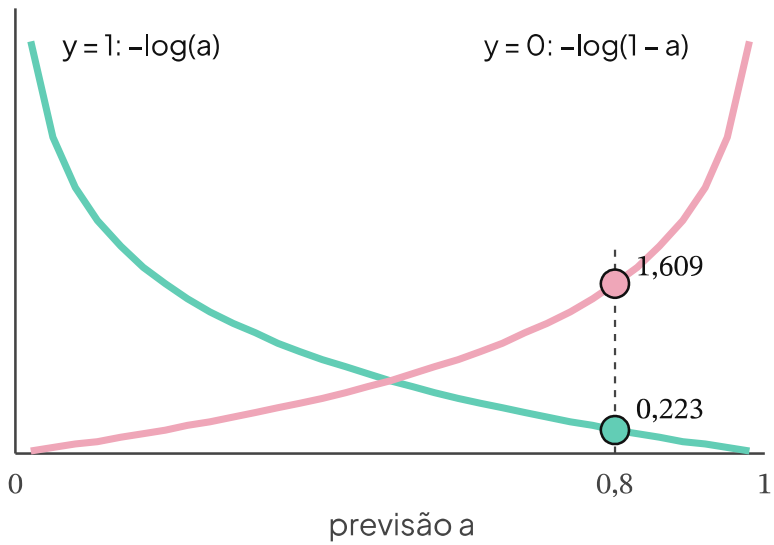
y = 0 · a = 0,8

$$L = -\log(0{,}2) \approx 1{,}609$$

Errar com confiança alta custa mais de sete vezes mais.

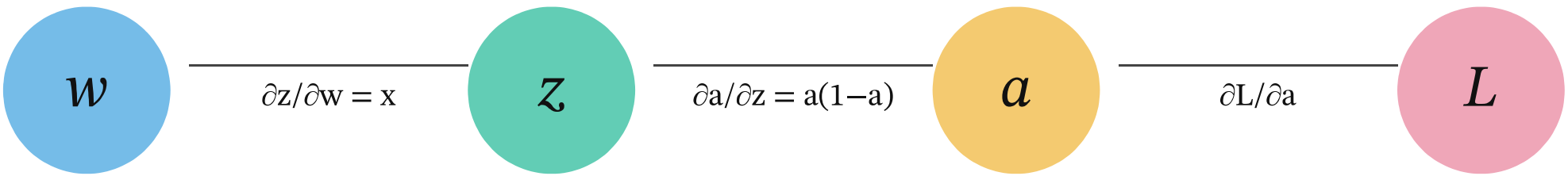
```python
def calcular_loss(Y, probabilidades):
    epsilon = 1e-12
    probabilidades = np.clip(
        probabilidades, epsilon, 1 - epsilon
    )
    loss = -np.mean(
        Y * np.log(probabilidades)
        + (1 - Y) * np.log(1 - probabilidades)
    )
    return float(loss)
```

epsilon impede log(0) = −∞ quando a previsão chega a 0 ou 1.



BLOCO 23 · DERIVADA COMPOSTA

Regra da cadeia



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Uma pequena variação Δw muda z, que muda a, que muda L: $\Delta L \approx (\partial L/\partial a)(\partial a/\partial z)(\partial z/\partial w)\,\Delta w$.

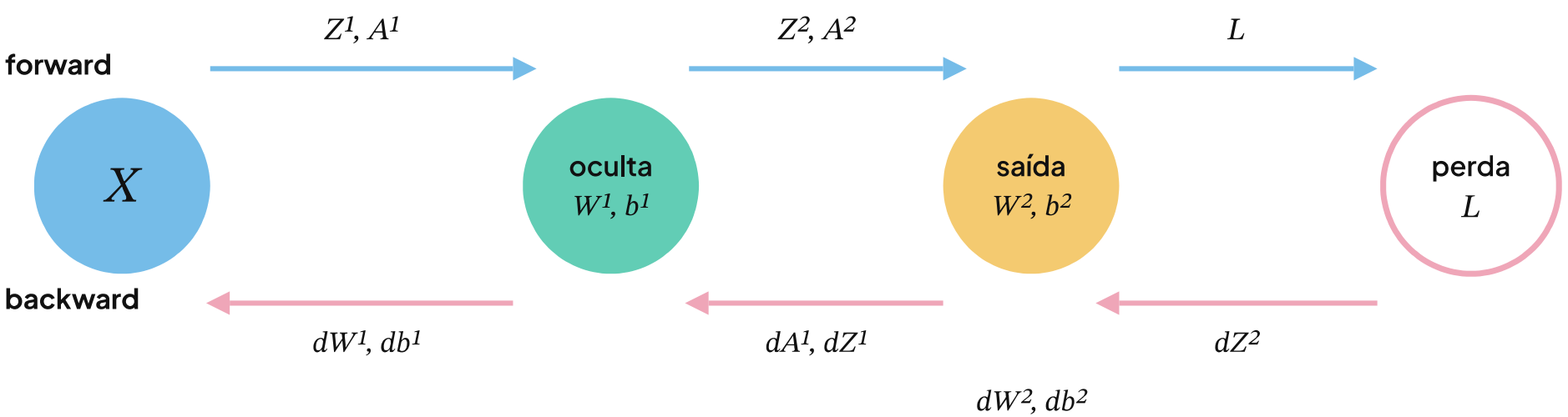
06

SEÇÃO 6 · 15 MINUTOS

Backpropagation e gradientes

A regra da cadeia aplicada à rede inteira, da saída para a entrada.

Forward e backward na rede



Forward calcula a previsão. Backpropagation calcula como cada parâmetro contribuiu para o erro.

BLOCO 24 · DERIVAÇÃO PARA UMA AMOSTRA

Gradiente na saída: sigmoid + BCE

1	$\partial \ell / \partial a = -\frac{y}{a} + \frac{1-y}{1-a} = \frac{a-y}{a(1-a)}$	derivada da perda
2	$\partial a / \partial z = a(1-a)$	derivada da sigmoid
3	$\partial \ell / \partial z = \frac{a-y}{\cancel{a(1-a)}} \cdot \cancel{a(1-a)} = a - y$	regra da cadeia: o fator se cancela
4	$dZ^{[2]} = \frac{A^{[2]} - Y}{m}$	média sobre m amostras

Válido para sigmoid + BCE com perda média. Outras combinações de ativação e perda dão outras expressões.

BLOCO 25 · CAMADA DE SAÍDA

Gradientes da camada de saída

$$dW^{[2]} = (A^{[1]})^{T}\, dZ^{[2]}$$

$4 \times m \cdot m \times 1 = 4 \times 1$

$$dW^{[2]}{}_{j} = \sum_i a^{[1]}{}_{ij} \cdot dZ^{[2]}{}_{i}$$

Cada peso recebe o erro da saída multiplicado pela ativação oculta que ele transporta, somado sobre as amostras.

$$db^{[2]} = \sum_i dZ^{[2]}{}_{i}$$

$m \times 1 \rightarrow$ soma das linhas $\rightarrow 1 \times 1$

$$\partial z^{[2]}{}_{i} / \partial b^{[2]} = 1$$

O bias afeta todas as amostras igualmente, então o seu gradiente é a soma dos gradientes da saída.

```
dW2 = A1.T @ dZ2                                # (4, 1)
db2 = np.sum(dZ2, axis=0, keepdims=True)        # (1, 1)
```

BLOCO 26 · DO ERRO DA SAÍDA PARA A CAMADA OCULTA

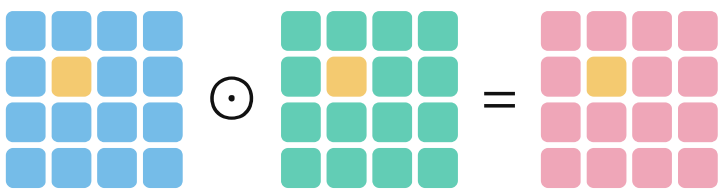
Retropropagação para a camada oculta

$$dA^{[1]} = dZ^{[2]}(W^{[2]})^{T}$$

$\mathrm{m} \times 1 \cdot 1 \times 4 = \mathrm{m} \times 4$

Cada neurônio oculto recebe uma parcela do erro proporcional ao peso que o liga à saída.

$$dZ^{[1]} = dA^{[1]} \odot A^{[1]}(1 - A^{[1]})$$



Hadamard (⊙): produto elemento a elemento. A posição (i, j) só usa a posição (i, j) de cada matriz. A1(1 – A1) é a derivada da sigmoid.

```
dA1 = dZ2 @ params["W2"].T      # (m, 4)  produto matricial
dZ1 = dA1 * A1 * (1 - A1)        # (m, 4)  elemento a elemento
```

BLOCO 27 · CAMADA OCULTA

Gradientes da primeira camada

$$dW^{[1]} = X^{T}\, dZ^{[1]}$$

$2 \times m \cdot m \times 4 = 2 \times 4$

$$db^{[1]} = \sum_i dZ^{[1]}{}_i$$

$m \times 4 \rightarrow$ soma das linhas $\rightarrow 1 \times 4$

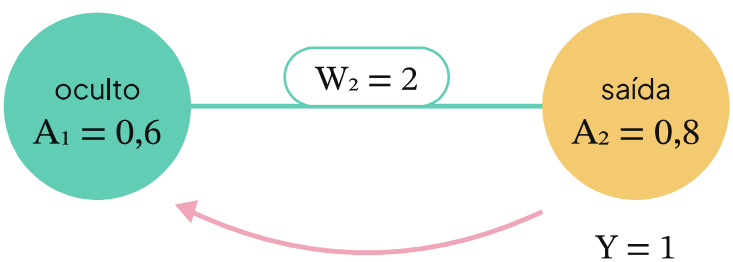
```
dW1 = X.T @ dZ1                              # (2, 4)
db1 = np.sum(dZ1, axis=0, keepdims=True)     # (1, 4)
```

Parâmetro	Gradiente	Formato
$W1$	$dW1$	2×4
$b1$	$db1$	1×4
$W2$	$dW2$	4×1
$b2$	$db2$	1×1

Todo gradiente tem o mesmo formato do parâmetro correspondente.

EXEMPLO DE UMA ÚNICA AMOSTRA

Exemplo numérico do gradiente retornando



o erro volta pelo peso

1	$dZ_2 = A_2 - Y = 0{,}8 - 1$	$-0{,}2$
2	$dW_2 = A_1 \cdot dZ_2 = 0{,}6 \cdot (-0{,}2)$	$-0{,}12$
3	$dA_1 = dZ_2 \cdot W_2 = -0{,}2 \cdot 2$	$-0{,}4$
4	$dZ_1 = -0{,}4 \cdot 0{,}6 \cdot 0{,}4$	$-0{,}096$

dW_2 negativo: $W \leftarrow W - \eta \cdot dW$ aumenta o peso e aproxima a saída de 1.

BLOCO 28 · ATUALIZAÇÃO DOS PESOS

Gradient descent e taxa de aprendizado

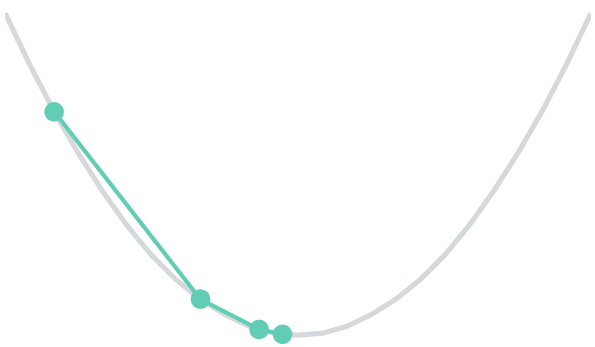
$$W \leftarrow W - \eta\, dW \qquad b \leftarrow b - \eta\, db$$

O gradiente aponta para onde a perda cresce; o sinal de menos leva o passo para o sentido oposto.



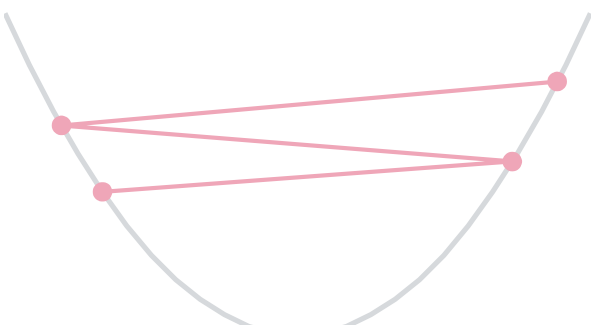
η = 0,05 · muito baixa

passos curtos, descida lenta



η = 0,3 · adequada

chega perto do mínimo em poucos passos



η = 1,05 · muito alta

oscila e se afasta do mínimo

Exemplo: $L(w) = w^2$, gradiente $2w$. Cada passo multiplica w por $(1 - 2\eta)$.

BLOCO 30 · IMPLEMENTAÇÃO

Backward em NumPy

```python
def backward(Y, params, cache):
    m = Y.shape[0]
    X = cache["X"]
    A1 = cache["A1"]
    A2 = cache["A2"]
    dZ2 = (A2 - Y) / m
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)
    dA1 = dZ2 @ params["W2"].T
    dZ1 = dA1 * A1 * (1 - A1)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)
    return {
        "W1": dW1,
        "b1": db1,
        "W2": dW2,
        "b2": db2
    }
```

$dZ^2 = (A^2 - Y)/m$	$m \times 1$
$dW^2 = (A^1)^T\, dZ^2$	4×1
$db^2 = \sum dZ^2$	1×1
$dA^1 = dZ^2\, (W^2)^T$	$m \times 4$
$dZ^1 = dA^1 \odot A^1(1 - A^1)$	$m \times 4$
$dW^1 = X^T\, dZ^1$	2×4
$db^1 = \sum dZ^1$	1×4

W2 é usado antes de qualquer atualização: todos os gradientes são calculados primeiro.

BLOCOS 29 E 31 · TREINAMENTO FULL BATCH

Atualização e loop de treinamento

```python
def update_parameters(params, gradients, learning_rate):
    for key in params:
        params[key] -= learning_rate * gradients[key]

    return params
```

```python
learning_rate = 0.5
epochs = 20000
historico_loss = []

for epoca in range(epochs):
    A2, cache = forward(X, params)
    loss = calcular_loss(Y, A2)
    gradients = backward(Y, params, cache)
    params = update_parameters(params, gradients, learning_rate)

    if epoca % 100 == 0:
        historico_loss.append(loss)
```

1. Forward
2. Perda
3. Backward
4. Atualização

Época: uma passagem pelos 4 casos do XOR.

Full batch: os 4 casos juntos, uma atualização por época.

Histórico: perda a cada 100 épocas, 200 valores.

07

SEÇÃO 7 · 30 MINUTOS

Exercício guiado em NumPy

Um notebook vazio. Uma rede neural que aprende o XOR.

`Aula_6_12_Exercicio_Guiado_Perceptron_MLP.ipynb`

EXERCÍCIO · ETAPAS 1, 2 E 3

Imports, tabela XOR e visualização

```python
# Etapa 1
import numpy as np
import matplotlib.pyplot as plt
# Etapa 2
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)
Y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)
print("Formato de X:", X.shape)
print("Formato de Y:", Y.shape)
```

```
Formato de X: (4, 2)
Formato de Y: (4, 1)
```

```python
# Etapa 3
plt.figure(figsize=(5, 5))
plt.scatter(X[:, 0], X[:, 1], c=Y.ravel(),
            cmap="coolwarm", s=300,
            edgecolors="black")
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("XOR: classes em diagonais opostas")
plt.show()
```

dtype=float · pesos reais e exponenciais

Y em 4 × 1 · mesmo formato de A2; com (4,), A2 – Y viraria 4 × 4

c=Y.ravel() · cor de cada ponto pela classe

EXERCÍCIO · ETAPAS 4, 5, 6 E 7

Perceptron no OR e no XOR

```
# Etapa 4
Y_or = np.array([-1, 1, 1, 1])

# Etapa 5: definir treinar_perceptron
# (mesmo código do slide 15)

# Etapa 6
pesos, bias, epocas = treinar_perceptron(X, Y_or)
z = X @ pesos + bias
prev_or = np.where(z >= 0, 1, -1)
print(pesos, bias, epocas)
print(prev_or, Y_or)
```

Rastreamento manual = execução de referência

```
[0.1 0.1] -0.1 4
[-1  1  1  1] [-1  1  1  1]
```

```
# Etapa 7
Y_xor_pm = np.array([-1, 1, 1, -1])
p_x, b_x, ep_x = treinar_perceptron(X, Y_xor_pm)
prev_xor = np.where(X @ p_x + b_x >= 0, 1, -1)
print(ep_x)
print(prev_xor, Y_xor_pm)
```

ep_x = 100 = max_epocas

Nenhuma época termina sem erros. Pela prova algébrica, nenhum w e b classifica as quatro linhas.

Execução de referência

```
100
[ 1  1 -1 -1] [-1  1  1 -1]
```

2 de 4 acertos · w = [-0.1 0.], b = 0.0

EXERCÍCIO · ETAPAS 8, 9, 10 E 11

Sigmoid, parâmetros, forward e perda inicial

```python
# Etapa 8: definir sigmoid (slide 30)

# Etapa 9
rng = np.random.default_rng(42)

n_entradas = 2
n_ocultos = 4
n_saidas = 1

params = {
    "W1": rng.normal(0, 0.8, size=(n_entradas, n_ocultos)),
    "b1": np.zeros((1, n_ocultos)),
    "W2": rng.normal(0, 0.8, size=(n_ocultos, n_saidas)),
    "b2": np.zeros((1, n_saidas))
}
for k, v in params.items():
    print(k, v.shape)

# Etapas 10 e 11: definir forward e calcular_loss
A2_inicial, _ = forward(X, params)
loss_inicial = calcular_loss(Y, A2_inicial)
```

```
W1 (2, 4)
b1 (1, 4)
W2 (4, 1)
b2 (1, 1)
```

Previsões iniciais A2

[0.578 0.613 0.660 0.682]

execução de referência · semente 42

Perda inicial

0.7285 (acima de log 2 ≈ 0.693)

Semente 42: mesmos pesos iniciais para todos. Pesos aleatórios quebram a simetria entre neurônios.

EXERCÍCIO · ETAPAS 12 A 16

Backward, atualização e treinamento

```python
# Etapa 12: exemplo da regra da cadeia (slide 41)
# Etapa 13: definir backward (slide 43)
grads = backward(Y, params, forward(X, params)[1])
for k in grads:
    print(k, grads[k].shape, params[k].shape)

# Etapa 14: definir update_parameters (slide 44)

# Etapas 15 e 16
learning_rate = 0.5
epochs = 20000
historico_loss = []

for epoca in range(epochs):
    A2, cache = forward(X, params)
    loss = calcular_loss(Y, A2)
    gradients = backward(Y, params, cache)
    params = update_parameters(params, gradients, learning_rate)
    if epoca % 100 == 0:
        historico_loss.append(loss)

print("Perda inicial:", historico_loss[0])
print("Perda final:", calcular_loss(Y, forward(X, params)[0]))
```

```
W1 (2, 4) (2, 4)
b1 (1, 4) (1, 4)
W2 (4, 1) (4, 1)
b2 (1, 1) (1, 1)
```

Perda inicial e final

0.7285 → 0.0007

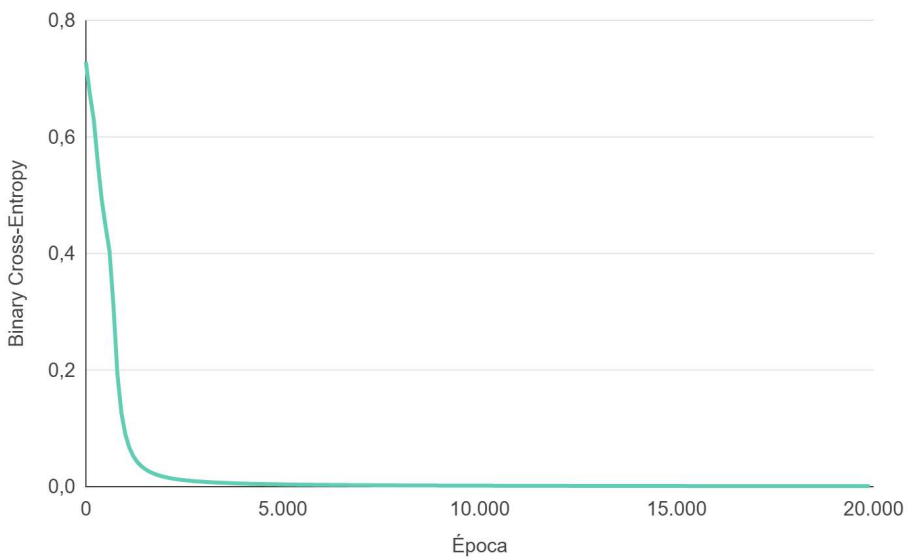
execução de referência · semente 42

Sem divisão treino/teste: o objetivo é aprender e verificar a tabela lógica completa.

EXERCÍCIO · ETAPAS 17 E 18

Curva de perda e previsões finais

```
epocas_reg = np.arange(len(historico_loss)) * 100
plt.plot(epocas_reg, historico_loss)
plt.xlabel("Época")
plt.ylabel("Binary Cross-Entropy")
plt.show()
```




```
probabilidades, _ = forward(X, params)
previsoes_binarias = (probabilidades >= 0.5).astype(int)
print(previsoes_binarias.ravel())
```

x	A2	classe	esperado
(0; 0)	0.0002	**0**	0
(0; 1)	0.9994	**1**	1
(1; 0)	0.9991	**1**	1
(1; 1)	0.0013	**0**	0

Previsões binárias: **[0 1 1 0]**. A rede aprendeu as quatro linhas do XOR com limiar 0,5.

EXERCÍCIO · ETAPA 19

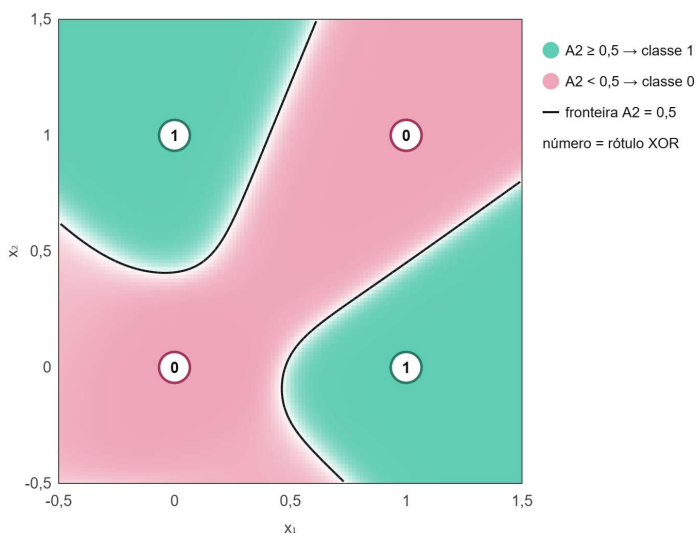
Região de decisão da rede treinada

```
x1 = np.linspace(-0.5, 1.5, 300)
x2 = np.linspace(-0.5, 1.5, 300)
G1, G2 = np.meshgrid(x1, x2)               # (300, 300)

grade = np.column_stack([G1.ravel(), G2.ravel()])  # (90000, 2)
saida, _ = forward(grade, params)                  # (90000, 1)
saida = saida.reshape(G1.shape)                    # (300, 300)

plt.contourf(G1, G2, saida, levels=20, cmap="coolwarm")
plt.contour(G1, G2, saida, levels=[0.5], colors="black")
plt.scatter(X[:, 0], X[:, 1], c=Y.ravel(), cmap="coolwarm",
            s=200, edgecolors="black")
plt.show()
```

forward foi escrito para m amostras: a grade de 90 000 pontos passa sem nenhuma alteração.



Código consolidado da MLP

```python
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[0, 0], [0, 1],
              [1, 0], [1, 1]], dtype=float)
Y = np.array([[0], [1], [1], [0]], dtype=float)


def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))
```

```python
def forward(X, params):
    Z1 = X @ params["W1"] + params["b1"]
    A1 = sigmoid(Z1)
    Z2 = A1 @ params["W2"] + params["b2"]
    A2 = sigmoid(Z2)
    cache = {"X": X, "Z1": Z1, "A1": A1,
             "Z2": Z2, "A2": A2}
    return A2, cache


def calcular_loss(Y, probabilidades):
    eps = 1e-12
    p = np.clip(probabilidades, eps, 1 - eps)
    loss = -np.mean(Y * np.log(p)
                    + (1 - Y) * np.log(1 - p))
    return float(loss)
```

Código consolidado da MLP

```python
def backward(Y, params, cache):
    m = Y.shape[0]
    X, A1 = cache["X"], cache["A1"]
    A2 = cache["A2"]
    dZ2 = (A2 - Y) / m
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)
    dA1 = dZ2 @ params["W2"].T
    dZ1 = dA1 * A1 * (1 - A1)
    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)
    return {"W1": dW1, "b1": db1,
            "W2": dW2, "b2": db2}


def update_parameters(params, gradients, lr):
    for key in params:
        params[key] -= lr * gradients[key]
    return params
```

```python
rng = np.random.default_rng(42)
params = {
    "W1": rng.normal(0, 0.8, size=(2, 4)),
    "b1": np.zeros((1, 4)),
    "W2": rng.normal(0, 0.8, size=(4, 1)),
    "b2": np.zeros((1, 1)),
}

historico_loss = []
for epoca in range(20000):
    A2, cache = forward(X, params)
    loss = calcular_loss(Y, A2)
    grads = backward(Y, params, cache)
    params = update_parameters(params, grads, 0.5)
    if epoca % 100 == 0:
        historico_loss.append(loss)

prob, _ = forward(X, params)
print((prob >= 0.5).astype(int).ravel())
```

EXERCÍCIO · ETAPA 20

Perceptron e MLP comparados

Perceptron

Um único neurônio

Fronteira linear

Regra de atualização própria

Não resolve XOR

MLP

Camada oculta e representações intermediárias

Funções diferenciáveis

Backpropagation

Aprende XOR

A camada oculta leva os dados a um novo espaço em que uma fronteira linear basta; no espaço original, essa fronteira é curva.

08

SEÇÃO 8 · 5 MINUTOS E MATERIAL DE APOIO

Erros comuns, quiz e desafio

Revisar, verificar e continuar praticando.

ERROS COMUNS · 1 DE 3

Erros comuns de conceito

1 **Confundir peso com bias**

Peso multiplica uma entrada e gira a fronteira; bias é somado e a desloca.

2 **Ativação e perda como a mesma coisa**

Ativação transforma z em cada neurônio; perda mede o erro da rede inteira.

3 **Confundir Perceptron com MLP**

Perceptron: um neurônio, degrau, regra própria. MLP: camadas ocultas, sigmoid, Backpropagation.

4 **Perceptron resolve qualquer classificação**

Só dados linearmente separáveis; apenas nesse caso a convergência é garantida.

5 **Achar que XOR é linearmente separável**

As quatro condições levam a $w_1 + w_2 + b > 0$ e < 0 ao mesmo tempo.

6 **Contar a entrada como camada treinável**

A rede $2 \to 4 \to 1$ tem duas camadas com parâmetros: oculta e saída.

Erros comuns de matemática

7 **Errar as dimensões**

(a × k)(k × c) = (a × c). W1 é 2 × 4: linha = entrada, coluna = neurônio.

8 **Esquecer o bias**

Sem b, toda fronteira passa pela origem. Cada camada soma o seu bias.

9 **Sigmoid sem a derivada**

O backward usa σ'(z) = a(1 – a) em cada camada sigmoid.

10 **Regra da cadeia fora de ordem**

Da perda para trás: dZ2 → dW2, db2 → dA1 → dZ1 → dW1, db1.

11 **Confundir dA com dZ**

dA: gradiente na saída do neurônio. sc-camel-d-z = dA ⊙ σ'(Z): na entrada da ativação.

12 **Esquecer a média das amostras**

A perda é média, logo sc-camel-d-z2 = (A2 – Y) / m.

Erros comuns de código

13 Sinal errado na atualização

```
✗ params[k] += lr * g[k]
✓ params[k] -= lr * g[k]
```

14 Atualizar antes de todos os gradientes

dA1 usa W2 original. Primeiro backward completo, depois update.

15 Dados 1D onde se espera matriz

```
✗ Y.shape (4,) → A2 - Y: (4, 4)
✓ Y.shape (4, 1)
```

16 Probabilidades tratadas como classes

```
✓ (A2 >= 0.5).astype(int)
```

17 Degrau no Backpropagation

Derivada zero zera os gradientes. Use uma ativação diferenciável.

18 Mais épocas sempre resolvem

Taxa inadequada, saturação ou má inicialização não se corrigem só com tempo.

QUIZ · 1 DE 2 · GABARITO NAS NOTAS

Quiz: perguntas 1 a 8

1 O que é um neurônio artificial?

2 Qual a função dos pesos?

3 O que é bias?

4 O que faz uma função de ativação?

5 Como o Perceptron aprende?

6 O que significa separabilidade linear?

7 Por que o Perceptron não resolve XOR?

8 O que é uma MLP?

QUIZ · 2 DE 2 · GABARITO NAS NOTAS

Quiz: perguntas 9 a 16

9 O que é uma camada oculta?

10 Qual a diferença entre forward e backward?

11 O que é a função sigmoid?

12 O que é Binary Cross-Entropy?

13 Para que serve a regra da cadeia?

14 O que representa um gradiente?

15 Como ocorre a atualização dos pesos?

16 Por que usamos NumPy no lugar de uma biblioteca pronta nesta aula?

DESAFIO FINAL

Construindo uma rede neural do zero

1. Perceptron para AND
2. Mostrar por que ele não resolve XOR
3. Construir uma MLP em NumPy
4. Implementar forward
5. Implementar a função de perda
6. Implementar Backpropagation
7. Treinar a rede
8. Visualizar a curva de perda
9. Verificar as previsões
10. Variar os neurônios ocultos
11. Variar a taxa de aprendizado
12. Explicar como a rede aprendeu

Entregas

Notebook completo · código organizado · curva de perda · previsões · explicação matemática · conclusão

`Desafio_Alunos_TODO.ipynb`

`Desafio_Gabarito.ipynb`

AULA 6.12 · SÍNTESE

Como uma rede neural aprende

1. Neurônio
2. Perceptron
3. Limitação linear
4. XOR
5. MLP
6. Forward
7. Perda
8. Regra da cadeia
9. Backpropagation
10. Gradiente
11. Atualização
12. **Aprendizado**